Aula 23 - Construindo um Ecossistema MQTT com Node.js

Descrição: Esta aula aborda a arquitetura orientada a eventos do Node.js aplicada a sistemas distribuídos de mensageria leve (IoT e telemetria) utilizando o protocolo MQTT. Exploraremos o ecossistema com base no modelo assíncrono, buffers binários, EventEmitter e gerenciamento modular via npm.

---

1. Cronograma da Aula

Bloco	Duração	Conteúdo Programático
Bloco 1	45 min	Fundamentos: Node.js assíncrono, Event Loop, Buffers e Introdução ao MQTT vs HTTP/REST
Bloco 2	45 min	Construção do Broker MQTT Local com Node.js (`aedes` e `net`)
Bloco 3	45 min	Implementação do Publisher (Simulador de Sensores IoT) e Subscriber (Cliente Monitor)
Bloco 4	30 min	Simulação, Inspeção de Tráfego e Troca de Mensagens em Tempo Real
Bloco 5	45 min	Atividade Prática Dirigida: Sistema de Alarme Crítico com Tópicos Dinâmicos
Fechamento	15 min	Discussão de arquitetura, boas práticas e encerramento

---

2. Fundamentação Teórica

2.1 Node.js e Arquitetura Orientada a Eventos

Conforme visto nos fundamentos do runtime do Node.js:

- **Event Loop & Single-Thread:** O Node.js opera sob uma única thread principal, delegando operações de entrada e saída (I/O de rede e disco) para o sistema de forma não bloqueante (`non-blocking`).
- **Buffers:** No transporte de mensagens de rede, os dados trafegam nativamente sob a forma de `Buffer` (sequências de octetos). O MQTT manipula payloads binários compactos, garantindo eficiência em dispositivos com recursos limitados.
- **EventEmitter:** A classe base para a maioria das APIs de rede (incluindo instâncias de sockets TCP do módulo nativo `net` e clientes de mensagens) expõe eventos como `.on('connect')`, `.on('message')`, `.on('data')` e `.on('error')`.

2.2 HTTP/REST vs. MQTT (Pub/Sub)

- **HTTP/REST (Request-Response):** Modelo síncrono ou baseado em polling, stateless, centrado em URIs e verbos (`GET`, `POST`, `PUT`, `DELETE`). Alto overhead de cabeçalhos por pacote.
- **MQTT (Publish/Subscribe):** Arquitetura desacoplada no tempo e no espaço. Os clientes não se comunicam diretamente; eles se conectam a um **Broker central**, publicando ou assinando mensagens estruturadas por **Tópicos hierárquicos** (ex: `sensores/temperatura/sala1`).

```
None
+--------------------+               +------------------+
|  Publisher (Node)  | --- Publish ->|                  |
| (Sensor Simulado)  |    (Tópico)   |                  |
+--------------------+               |   MQTT Broker    |
                                     |    (Node.js)     |
+--------------------+               |                  |
|  Subscriber (Node) | <-- Deliver --|                  |
|  (Painel Monitor)  |  (Assinatura) +------------------+
+--------------------+
```

---

3. Preparação do Ambiente de Laboratório

Abra o terminal e crie uma pasta exclusiva para o ecossistema:

```shell
mkdir ecossistema-mqtt-node
cd ecossistema-mqtt-node
npm init -y
```

Instale as dependências essenciais:

- `aedes`: Engine broker MQTT de alto desempenho escrita em JavaScript/Node.js.
- `mqtt`: Biblioteca cliente MQTT universal compatível com Node.js e navegadores.

```shell
npm install aedes mqtt
```

---

4. Passo a Passo do Desenvolvimento

4.1 Construção do Broker MQTT (`broker.js`)

O broker é o servidor central responsável por autenticar clientes, receber publicações e rotear os pacotes aos respectivos assinantes. Utilizaremos o módulo nativo `net` do Node.js em conjunto com o motor `aedes`.

Crie o arquivo `broker.js`:

```javascript
// broker.js
const aedes = require('aedes')();
const net = require('net');

const PORT = 1883;

// Cria o servidor TCP utilizando o módulo nativo 'net'
const server = net.createServer(aedes.handle);
```

```
server.listen(PORT, function () {
  console.log(`[BROKER] Servidor MQTT ativo e escutando na porta ${PORT}`);
});

// Eventos de ciclo de vida do broker (EventEmitter)
aedes.on('client', function (client) {
  console.log(`[BROKER] Novo cliente conectado: ID = ${client.id}`);
});

aedes.on('clientDisconnect', function (client) {
  console.log(`[BROKER] Cliente desconectado: ID = ${client.id}`);
});

aedes.on('subscribe', function (subscriptions, client) {
  subscriptions.forEach((sub) => {
    console.log(`[BROKER] Cliente ${client.id} assinou o tópico: ${sub.topic}`);
  });
});

aedes.on('publish', function (packet, client) {
  // Ignora pacotes internos de controle da engine
  if (client) {
    console.log(`[BROKER] Mensagem roteada de ${client.id} -> Tópico: ${packet.topic} | Payload:
${packet.payload.toString()}`);
  }
});
```

4.2 Construção do Subscriber (`subscriber.js`)

O assinante estabelece conexão com o broker e registra interesse em tópicos específicos usando coringas (`#` para múltiplos níveis ou `+` para nível único).

Crie o arquivo `subscriber.js`:

JavaScript

```
// subscriber.js
const mqtt = require('mqtt');

const BROKER_URL = 'mqtt://127.0.0.1:1883';
const CLIENT_ID = 'central_monitoramento_01';

const client = mqtt.connect(BROKER_URL, {
  clientId: CLIENT_ID,
  clean: true
});

client.on('connect', function () {
  console.log(`[SUBSCRIBER] Conectado ao broker como '${CLIENT_ID}'`);
```

```
  // Assina todos os sensores sob a hierarquia 'industria/fabrica1/#'
  const topico = 'industria/fabrica1/#';
  client.subscribe(topico, function (err) {
    if (!err) {
      console.log(`[SUBSCRIBER] Assinatura ativa para: ${topico}`);
    } else {
      console.error('[SUBSCRIBER] Falha na assinatura:', err);
    }
  });
});

// O payload recebido é um Buffer do Node.js
client.on('message', function (topic, payload) {
  try {
    const rawData = payload.toString();
    const dados = JSON.parse(rawData);

    console.log(`\n---------------- NOVA LEITURA RECEBIDA ----------------`);
    console.log(`Tópico de Origem: ${topic}`);
    console.log(`Dispositivo ID   : ${dados.sensorId}`);
    console.log(`Temperatura      : ${dados.temperatura} °C`);
    console.log(`Umidade          : ${dados.umidade} %`);
    console.log(`Timestamp        : ${dados.timestamp}`);
    console.log(`-------------------------------------------------------`);
  } catch (error) {
    console.error('[SUBSCRIBER] Erro ao decodificar payload JSON:', error.message);
  }
});

client.on('error', function (err) {
  console.error('[SUBSCRIBER] Erro de conexão:', err);
});
```

4.3 Construção do Publisher Simulado (`publisher.js`)

O publicador simula um dispositivo industrial enviando telemetria em intervalos contínuos via `setInterval()`.

Crie o arquivo `publisher.js`:

JavaScript

```
// publisher.js
const mqtt = require('mqtt');

const BROKER_URL = 'mqtt://127.0.0.1:1883';
const SENSOR_ID = 'sensor_temp_01';
const TOPIC = 'industria/fabrica1/temperatura';

const client = mqtt.connect(BROKER_URL, {
  clientId: SENSOR_ID
});
```

```
client.on('connect', function () {
  console.log(`[PUBLISHER] Sensor '${SENSOR_ID}' conectado com sucesso.`);

  // Temporizador assíncrono emitindo telemetria a cada 3 segundos
  setInterval(() => {
    const temperatura = parseFloat((20 + Math.random() * 15).toFixed(2));
    const umidade = parseFloat((40 + Math.random() * 30).toFixed(2));

    const payload = JSON.stringify({
      sensorId: SENSOR_ID,
      temperatura: temperatura,
      umidade: umidade,
      timestamp: new Date().toISOString()
    });

    // client.publish aceita strings ou Buffers do Node
    client.publish(TOPIC, payload, { qos: 0 }, (err) => {
      if (err) {
        console.error(`[PUBLISHER] Erro ao enviar pacote:`, err);
      } else {
        console.log(`[PUBLISHER] Enviado para [${TOPIC}]: ${payload}`);
      }
    });
  }, 3000);
});

client.on('error', function (err) {
  console.error('[PUBLISHER] Erro de conexão:', err);
});
```

---

5. Roteiro de Execução e Simulação

Para validar a troca de dados em tempo real, oriente os alunos a abrir **três terminais distintos**:

1. **Terminal 1 (Broker):**

```
Shell
node broker.js
```

   Saída esperada: `[BROKER] Servidor MQTT ativo e escutando na porta 1883`

2. **Terminal 2 (Subscriber):**

```
Shell
node subscriber.js
```

   Saídas esperadas:

- No Terminal 1: `[BROKER] Novo cliente conectado: ID = central_monitoramento_01` e confirmação da assinatura.
- No Terminal 2: `[SUBSCRIBER] Assinatura ativa para: industria/fabrica1/#`

3. **Terminal 3 (Publisher):**

```
Shell
node publisher.js
```

Saídas esperadas:

- Os três terminais exibirão o tráfego coordenado em tempo real: envio pelo publisher, roteamento pelo broker e exibição formatada no subscriber.

---

6. Atividade Prática Dirigida (Hands-On)

Desafio: Sistema de Alerta com Atuador Crítico

Tempo Estimado: 45 minutos
Modo de trabalho: Individual.

Descrição do Problema

Você deve expandir o ecossistema com regras de negócio e controle reativo:

1. **Publisher - Múltiplos Sensores:**
   - Crie uma variação do `publisher.js` que gere picos anormais de temperatura (valores acima de 30ºC).
2. **Subscriber Analítico (Detector de Anomalias):**
   - Modifique o `subscriber.js` para analisar o valor de `temperatura`.
   - Se temperatura > 30.0º C, o cliente deve agir como um **Publisher reativo** e disparar uma mensagem crítica no tópico: `industria/fabrica1/alarmes` Payload de exemplo:

```
JSON
{
  "alerta": "CRÍTICO",
  "temperatura": 32.5,
  "sensorId": "sensor_temp_01",
  "instrucao": "Ativar sistema de arrefecimento"
}
```

Cliente Atuador (Subscriber de Alarme):

- Crie um arquivo `actuator.js` que assine unicamente `industria/fabrica1/alarmes`.
- Ao receber o pacote de alerta, exiba em destaque no terminal: `[ATUADOR] RESFRIAMENTO DE EMERGÊNCIA ATIVADO!`.